

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA



BÁO CÁO BÀI TẬP LỚN
MÔN MẠNG MÁY TÍNH
Học kỳ 2 - Năm học 2025-2026
Mã môn học: CO3093/CO3094

**Đề tài: Assignment 1 - Implement non-blocking
HTTP server and chat application**

Giảng viên hướng dẫn: Thầy Bùi Xuân Giang

Thành viên thực hiện:

Phan Thành Long - 2211891

...

Thành Phố Hồ Chí Minh, năm 2026

Mục lục

I. GIỚI THIỆU ĐỀ TÀI VÀ KIẾN TRÚC HỆ THỐNG	2
1 Mục tiêu đề tài	2
2 Tổng quan kiến trúc AsynapRous	2
II. TRIỂN KHAI CƠ CHẾ NON-BLOCKING	3
1 Lựa chọn cơ chế Coroutine (AsyncIO)	3
2 Tích hợp vào HttpAdapter	3
III. CƠ CHẾ XÁC THỰC (AUTHENTICATION)	4
1 Cơ chế hoạt động của HTTP Cookies	4
2 Cài đặt trong hệ thống	4
IV. ỨNG DỤNG CHAT HYBRID (CLIENT-SERVER & P2P)	5
1 Giai đoạn khởi tạo (Client-Server Paradigm)	5
2 Giai đoạn truyền tin (P2P Paradigm)	5
V. TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN	6
1 Tổng kết	6
2 Hướng phát triển	6

I GIỚI THIỆU ĐỀ TÀI VÀ KIẾN TRÚC HỆ THỐNG

1 Mục tiêu đề tài

Bài tập lớn yêu cầu việc xây dựng một hệ thống HTTP Server vận hành theo cơ chế non-blocking, đồng thời phát triển một ứng dụng Chat kết hợp mô hình Client-Server và Peer-to-Peer (P2P). Mục tiêu của dự án là giúp sinh viên nắm vững các thành phần lõi trong lập trình mạng (Network Programming), cụ thể:

- Hiểu và vận hành các tiến trình máy chủ (Server processes) và proxy.
- Triển khai cơ chế Non-blocking TCP Communication để quản lý hàng nghìn kết nối đồng thời mà không bị treo (blocking) hệ thống.
- Xây dựng nền tảng WebApp (AsynapRous) hỗ trợ thiết kế RESTful API để định tuyến (routing) yêu cầu từ người dùng.

2 Tổng quan kiến trúc AsynapRous

Hệ thống sử dụng nền tảng AsynapRous (Asynchronous Application Routing), một lightweight framework do nhóm tự xây dựng để hỗ trợ triển khai các RESTful endpoint. Framework này cung cấp khả năng:

- Đăng ký các hàm xử lý qua Decorator `@app.route()` tương tự các framework hiện đại.
- Tích hợp chặt chẽ với cơ chế Non-blocking Coroutine (Sử dụng AsyncIO) để nâng cao hiệu suất giao tiếp giữa client và server.
- Thiết kế module linh hoạt gồm: Proxy Server, Backend Server và Application (SampleApp).

II TRIỂN KHAI CƠ CHẾ NON-BLOCKING

1 Lựa chọn cơ chế Coroutine (AsyncIO)

Trong lập trình socket TCP truyền thống (Blocking mode), phương thức `recv()` hoặc `send()` sẽ dừng toàn bộ tiến trình cho đến khi dữ liệu được truyền xong. Điều này gây lãng phí tài nguyên và không thể đáp ứng cho hàng nghìn request cùng lúc.

Để giải quyết, hệ thống đã ứng dụng cơ chế **Coroutine** dựa trên thư viện `asyncio` của Python.

- Các hàm xử lý kết nối từ client được định nghĩa dưới dạng `async def`.
- `StreamReader` và `StreamWriter` được sử dụng như các high-level API thay thế cho socket thuần túy để thực hiện đọc/ghi dữ liệu không đồng bộ.
- Event Loop của `asyncio` sẽ tự động điều phối, tạm dừng coroutine khi chờ I/O và chuyển sang thực thi tác vụ khác, tối ưu hóa tốc độ xử lý mạng.

2 Tích hợp vào HttpAdapter

Hàm `handle_client_coroutine` trong `HttpAdapter` được chịu trách nhiệm tiếp nhận và định tuyến:

```
1 async def handle_client_coroutine(self, reader, writer):
2     # c k h n g n g b y u c u t c l i e n t
3     request_data = await reader.read(4096)
4
5     # P h n t c h v k h i t o R e q u e s t o b j e c t
6     # G i h m x l (route handler) m t c c h b t
7     # P h n h i l i c h o c l i e n t s d n g w r i t e r . w r i t e ()
```

III CƠ CHẾ XÁC THỰC (AUTHENTICATION)

1 Cơ chế hoạt động của HTTP Cookies

Xác thực danh tính là một tính năng bắt buộc. Dự án đã áp dụng cơ chế quản lý trạng thái (State Management) thông qua **HTTP Cookies** (RFC 6265).

1. Khi client đăng nhập, máy chủ sẽ tạo một phiên (session) và gửi kèm thông tin về Cookie thông qua Header **Set-Cookie** trong **Response**.
2. Trình duyệt của client sẽ lưu trữ Cookie này và tự động gắn nó vào **Cookie Header** trong tất cả các request tiếp theo.

2 Cài đặt trong hệ thống

Trong module `request.py` và `httpadapter.py`, hệ thống đã xây dựng lớp `CaseInsensitiveDict` để dễ dàng đọc và quản lý Headers. Hàm `extract_cookies` có chức năng trích xuất toàn bộ chuỗi Cookie từ Request của client và lưu thành Dictionary, cho phép các RESTful API có thể truy xuất định danh người dùng một cách bảo mật.

IV ỨNG DỤNG CHAT HYBRID (CLIENT-SERVER & P2P)

Hệ thống Chat được xây dựng trên một kết cấu Hybrid kết hợp 2 giai đoạn chính:

1 Giai đoạn khởi tạo (Client-Server Paradigm)

Một Server trung tâm chịu trách nhiệm đóng vai trò làm Tracker để lưu trữ danh sách các điểm nút mạng (Peers). Các API được thiết kế trong AsynapRous:

- [POST] `/submit-info`: Khi một peer mới muốn tham gia, nó gửi IP và Port lên Tracker để đăng ký.
- [GET] `/get-list`: Các peer gọi API này để tải về danh sách các peer đang trực tuyến (`active_peers`).

2 Giai đoạn truyền tin (P2P Paradigm)

Khi một peer đã nắm được danh sách trực tuyến, nó có thể thiết lập kết nối trực tiếp đến một peer khác mà không cần đi qua Server trung tâm.

- [POST] `/broadcast-peer`: Chức năng gửi tin nhắn đến toàn bộ phòng chat. Hệ thống sẽ duyệt qua `active_peers` và gửi HTTP POST chứa nội dung tin nhắn trực tiếp đến các node.
- [POST] `/send-peer`: Chức năng nhắn tin trực tiếp riêng lẻ giữa hai node.

Việc sử dụng `asyncio` giúp các tác vụ Broadcast và nhận tin nhắn không làm đóng băng giao diện người dùng, đảm bảo trải nghiệm Chat mượt mà, thời gian thực.

V TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

1 Tổng kết

Bài tập lớn đã hoàn thiện việc tự xây dựng một mô hình HTTP Server tối ưu sử dụng cơ chế Non-blocking I/O. Các thành phần lõi như Proxy, Backend, HttpAdapter, và WebApp Framework (AsynapRous) được tổ chức theo nguyên lý Module hóa chặt chẽ. Hệ thống Chat Hybrid cũng chứng minh được tính hiệu quả trong việc duy trì kết nối phân tán và định tuyến RESTful tin cậy.

2 Hướng phát triển

Trong tương lai, hệ thống có thể được nâng cấp ở những khía cạnh sau:

- Tích hợp mã hóa bảo mật lớp Socket bằng SSL/TLS để triển khai giao thức HTTPS.
- Bổ sung Websocket để thay thế phương pháp polling (gọi API liên tục) trong hệ thống Chat, từ đó giảm thiểu độ trễ mạng và số lượng Request ảo.
- Cải thiện khả năng phân tải (Load Balancing) của Proxy từ cơ chế Round-robin cơ bản sang Least-connection hoặc dựa trên chỉ số phản hồi của Backend.